



ForgeGIS™

Technical Brief

VERSION 1.1 · JUNE 2026

Forged For Speed. Refined by Craft.

SEAGLASS FOUNDRY LLC

Executive Summary

ForgeGIS™ is a GPU-accelerated geospatial library with a native Model Context Protocol (MCP) server, distributed as pure-Java Maven artifacts with no first-party native code, no GDAL native bindings, and no separate native install step. It provides 239 user-facing operations spanning terrain, bathymetry and ocean, hydrology, RF and visibility analysis, spectral and machine learning, spatial geometry, filtering and morphology, raster management and I/O, routing, point cloud, and temporal analysis — all callable directly from Java for programmatic pipelines, and exposed to AI agents through a dual-surface MCP server. As of the validation period for this brief, we are not aware of another commercially-supported GPU-accelerated geospatial MCP server.

239

user-facing operations across 11 categories

237/239

agent-accessible via MCP; 152 on the pipeline registry

7.5×

geomean vs. GDAL CLI; no category excluded

28×

pipeline category geomean, GPU-resident, vs. GDAL chains

The library is built around two architectural commitments that distinguish it from extension-based approaches to GPU GIS. First, the raster data plane is GPU-resident across raster operations rather than round-tripping through host memory or disk between stages — the design choice that makes multi-step, agent-driven analysis viable at machine speed. In release 1.1 this pipeline became non-linear: a directed-acyclic-graph (DAG) execution engine runs branching workflows across both raster and vector stages in a single graph, with the raster data plane GPU-resident across it, and imports existing QGIS processing models directly. Second, the deployment artifact is a single shaded Java jar with no first-party native code and no GDAL native bindings. GPU dispatch uses the JOCL OpenCL bindings, whose JNI bridge ships bundled inside the JOCL jar and extracts transparently at runtime; the system OpenCL runtime required for GPU access is standard software bundled with GPU drivers. Compared to GDAL-based deployments, this eliminates the separate native library installation, GLIBC version-pinning, and per-platform shared-library distribution that has historically dominated deployment cycles in regulated environments.

Performance, June 2026 validation run

ForgeGIS's strongest results come from multi-stage GPU-resident pipelines, where the persistent data plane eliminates inter-stage I/O entirely: the PIPELINE category posts a geometric mean of 28.5× over equivalent sequential GDAL CLI chains (warm-minimum times), with individual chains reaching 56× at 4096². Across the full catalog, ForgeGIS exceeds GDAL CLI by a geometric mean of 7.5× (warm-mean; 9.4× warm-minimum) over a cohort of 225 head-to-head records with no category excluded — 206 wins. This cohort rule is deliberately inclusive: it is every operation with a GDAL CLI equivalent where both backends were timed, not a hand-selected subset.

Validation and coverage

The validation run executes 1,010 operation records. 779 of them exercise operations with no GDAL CLI equivalent — the analytic surface in terrain analytics, interpolation, ocean analysis, ML classification, viewshed, spatial-compute, and trajectory work that GDAL does not address and that has historically required separate tools. These operations are validated against domain-appropriate references: JTS for Cartesian predicates and topology, PostGIS for spheroidal computations on the WGS84 ellipsoid, GDAL for raster operations with direct equivalents, and analytic numerical

fixtures where no external oracle exists. Outputs are verified deterministic across repeated runs, so the numerical guarantees scripted workflows depend on remain intact when the same operations are invoked by an AI agent.

ForgeGIS represents a generational shift in how geospatial analysis is delivered — from analyst-centric workflows to AI-native pipelines that operate at machine speed without sacrificing the determinism regulated work requires. Seaglass Foundry is seeking strategic partners and distributors positioned to bring this technology to defense, intelligence, climate, and enterprise markets at scale.

What ForgeGIS Is

ForgeGIS is a geospatial compute substrate built for two audiences whose needs are converging: organizations that operate professional GIS pipelines today, and organizations that intend to expose geospatial analysis to AI agents tomorrow. It does not aspire to replace the analyst-facing GIS desktop. It replaces the layer underneath — the compute engine that runs the operations the analyst, the script, or the agent invokes.

The defining architectural commitment is that the raster data plane stays GPU-resident across raster operations. A multi-step pipeline — read a tile, reproject it, compute slope, derive viewshed, write a Cloud-Optimized GeoTIFF — runs as one GPU-resident chain rather than seven processes round-tripping through disk. This is the structural reason agent-driven multi-step analysis is viable at machine speed on ForgeGIS where it is not on a sequence of GDAL CLI invocations.

The library is distributed as standard Maven artifacts with no first-party native code, no GDAL native bindings, and no separate native install step. A single shaded jar drops into existing Maven builds. GPU dispatch uses the JOCL OpenCL bindings — JNI bridge bundled inside the JOCL jar, extracted transparently at runtime — and the system OpenCL runtime that ships with GPU drivers. JOCL is the OpenCL backend today, with Vulkan compute as the next backend; the execution model is deliberately vendor-neutral rather than CUDA-locked.

Architecture

ForgeGIS is organized around principles that distinguish it from extension-based approaches to GPU GIS.

GPU-resident pipeline

Raster data lands on the GPU once and stays there for the duration of a pipeline, moving through successive raster operations as device buffers rather than being materialized to host memory or disk between stages. Inter-op I/O is the dominant cost in CLI-orchestrated geospatial pipelines; eliminating it is the structural source of the pipeline-level speedups documented in the Performance section.

The pipeline made non-linear: DAG execution (new in 1.1)

Release 1.1 extends the GPU-resident pipeline from linear chains to arbitrary directed acyclic graphs. Three executors — for raster-only, vector-only, and mixed raster/vector graphs — run branching workflows in which a single stage can fan out to multiple consumers and multiple inputs can converge on one operation, all while raster intermediates stay resident on the device. This is the same data-plane discipline that produces the headline pipeline speedups, now applied to the non-linear workflow shapes real analysis actually takes.

A SavedPipeline v2 wire format represents these graphs with typed slot references, conditionals, and nested pipelines; linear chains continue to emit the v1 format unchanged. The mixed executor closes the full four-cell operation grid — raster → raster, vector → vector, raster → vector, and vector → raster — so a graph can rasterize a vector layer, run focal analysis, polygonize the result, and clip it against another vector layer without ever leaving the graph.

QGIS model import

A dedicated import module reads QGIS .model3 processing models and compiles them to ForgeGIS DAGs, with a compatibility-report mode that names any unsupported algorithm up front against a frozen format contract. Organizations with existing QGIS processing investments get those models running on the GPU-resident engine without rebuilding them — the migration on-ramp for a shop that already thinks in QGIS workflows. The expression layer is a clean-room QGIS Processing expression interpreter (independent lexer, parser, and evaluator over a whitelisted function set), implemented without consulting QGIS source.

Pure-Java Maven distribution

ForgeGIS itself contains no first-party native code. GPU dispatch is via the JOCL OpenCL bindings, which package their JNI bridge inside the JOCL jar and extract it transparently at runtime — no separate native install step, and no GDAL native bindings or other multi-vendor native libraries enter the build. The deployment artifact is a single shaded jar.

Dual-surface API for programmatic and agent use

Every operation in the catalog is a first-class Java method, callable directly from any JVM application. The same operations are exposed through a native MCP server in two complementary patterns: a pipeline DSL that composes operations into multi-step (and, in 1.1, non-linear) workflows agents construct dynamically, and 60 curated intent tools providing typed single-call endpoints for common analytic tasks. The MCP surface totals 68 tools — 60 intent tools, 5 primitive handle and session-management tools, and 3 pipeline-escape tools. Of the 239 user-facing operations, 152 are on the composable pipeline registry; 237 are reachable through the MCP server in total. Two operations are Java-only by architectural choice.

Profile-gated discovery (new in 1.1). The MCP server advertises a MINIMAL surface of 8 tools by default — the 5 primitive and 3 pipeline-escape tools — so an agent's tool list stays small and stays inside context and rate-limit budgets. The full 68-tool STANDARD surface is enabled with a single --profile standard flag or environment variable. Hidden tools remain callable by name; the profile governs discovery, not capability.

Numerical determinism preserved across surfaces

The MCP server dispatches to the same compute implementations exposed through the direct Java API. The MCP layer is a parameter-marshaling shim, not a parallel implementation — numerical outputs depend on the operation and its parameters, not the invocation surface. The deterministic guarantees scripted workflows depend on remain intact when the same operations are invoked by an AI agent, a precondition for adoption in regulated environments where audit and reproducibility requirements apply.

One-directional module structure

The library is structured around strict one-directional Maven module dependencies. The compute core has no dependencies on the I/O layer, the MCP server, or the validation harness. The I/O layer depends on the compute core

and a small set of vetted geospatial format libraries. The MCP server depends on the catalog and the I/O layer. Consumers who do not need MCP, or who do not need the I/O layer's format support, can depend on subsets of the library without pulling in unnecessary surface area.

Capability Catalog

ForgeGIS provides 239 user-facing operations organized into eleven categories. Each category below summarizes the surface; the complete enumerated catalog appears as Appendix A. Release 1.1 added 19 user-facing operations over 1.0, concentrated in the vector and raster ↔ vector surface.

Terrain. Slope, aspect, hillshade, curvature, roughness, topographic position index, terrain ruggedness, vector ruggedness measure, surface-area ratio, and related derivatives — the foundation of any DEM-driven analysis.

Bathymetry and Ocean. Bathymetric position index, benthic irradiance, depth uncertainty, isopach computation, sound-velocity profile, SOFAR-channel identification, and related ocean-specific derivatives, for littoral and underwater acoustic workflows.

Hydrology. Sink filling, flow direction, flow accumulation, stream-network extraction, stream order, topographic wetness index, and watershed delineation — the standard hydrological derivative chain, GPU-resident end to end.

Visibility and RF. Viewshed, batched multi-observer viewshed, intervisibility, Fresnel zone, insolation, horizon angle, shadow, and sky-view factor — the line-of-sight and propagation surface for defense, telecommunications, and solar analysis.

Spectral and Machine Learning. Kriging, inverse distance weighting, kernel density estimation, principal component analysis, minimum noise fraction, pansharpening, spectral angle mapper, random forest, DBSCAN and HDBSCAN clustering, K-means and maximum-likelihood classification, connected-component labeling, and related spectral and ML primitives.

Spatial Geometry. The largest category. Vector buffering and variable buffering, clipping, reprojection, geometry validation and simplification, boolean overlay (difference, intersection), dissolve, Voronoi polygons, convex hull, centroids, least-cost path, K-nearest-neighbors, field calculators and an expression interpreter, and fifteen trajectory-analysis operations. Topological relationships cover the full DE-9IM surface: the 8 OGC SFS named predicates, the 3 JTS extensions, Relate for user-supplied mask matching, and the raw 9-character intersection matrix per geometry-type pair through six matrix-computing operations. Validated against JTS for topology and analytic oracles for distance and trajectory metrics.

Filtering, Focal, and Morphology. Gaussian blur, median filter, majority filter, edge detection, generalized convolution, morphological operations, nibble, sieve, spike removal, no-data filling, and related focal and morphological transforms.

Raster Management, Vectorize, and I/O. Reprojection, resampling, clipping, translation, mosaicking, rasterization, polygonization, clip-raster-by-mask-layer, contour generation, overview construction, band math, map algebra, zonal statistics, and the I/O surface for GeoTIFF, COG, NetCDF, GeoPackage, and standard vector formats. A bundled EGM96 geoid — the first dataset ForgeGIS ships — makes vertical-datum conversion work out of the box.

Routing. Traveling salesman, multi-depot TSP, capacitated vehicle routing, time-windowed VRP, service-area allocation, distance-matrix computation, and nearest-neighbor tour construction — routing on GPU rather than a CPU-bound graph engine, with implications for fleet-scale and humanitarian logistics workloads.

Point Cloud. Point-cloud rasterization, ground filtering, classification, canopy height model derivation, and voxelization — point cloud as a substrate for raster derivation rather than an end-to-end LIDAR platform.

Temporal. Temporal statistics and temporal anomaly detection — a substrate for time-series workflows expressed as repeated invocations of the wider catalog.

Performance

All performance numbers in this section are drawn from the June 2026 validation run on the reference hardware specified in the Deployment section. The full benchmark methodology is documented in Appendix B. The headline cohort rule is inclusive by design: every operation with a GDAL CLI equivalent where both backends were timed, with no category excluded.

Multi-stage pipelines — the headline result

Where ForgeGIS's architecture produces its strongest results is in multi-stage pipelines, where the GPU-resident data plane eliminates inter-stage I/O entirely. The PIPELINE category posts a geometric mean of 28.5× over equivalent sequential GDAL CLI chains (warm-minimum), winning every head-to-head record in the cohort. The figures below are at the two production raster scales used for headline materials; the DAG chains are the new non-linear executors carrying real benchmark load.

Pipeline chain	Scale	GDAL warm-min	ForgeGIS warm-min	Speedup
AlgebraPipeline	1024 ²	204.70 ms	3.64 ms	56.27×
AlgebraPipeline	4096 ²	714.77 ms	19.82 ms	36.06×
TerrainPipeline	1024 ²	310.56 ms	20.88 ms	14.87×
TerrainPipeline	4096 ²	1,685.45 ms	185.08 ms	9.11×
DAG — variadic band-concat (new)	1024 ²	150.42 ms	12.37 ms	12.16×
DAG — linear slope/identity (new)	4096 ²	349.05 ms	32.65 ms	10.69×

Multi-stage pipeline chains vs. equivalent GDAL CLI chains. Warm-minimum times. June 2026 run.

These represent canonical multi-stage GIS workflows — raster-algebra compositions, terrain derivative chains, and the new non-linear DAG shapes — and are the operational form in which ForgeGIS's architectural decisions produce their largest measured advantage. The DAG executor results confirm that the non-linear engine carries the same performance characteristics as the linear pipeline it extends.

Catalog operations vs. GDAL CLI

Across the full inclusive cohort of 225 head-to-head records — every operation with a direct GDAL CLI equivalent where both backends were timed, no category excluded — ForgeGIS exceeds GDAL by a geometric mean of 7.5×

(warm-mean) and 9.4× (warm-minimum), winning 206 of 225. The two reported statistics differ only in whether each operation's warm series is summarized by its mean or its minimum; warm-mean is the more conservative headline and is used throughout this brief unless noted.

Category	Records	Geomean (warm-mean)	Median	Wins
Pipeline	14	26.83×	23.68×	14/14
Vectorize	43	12.65×	9.85×	36/43
Transform	10	10.14×	21.98×	10/10
Zonal / Focal	29	8.22×	11.50×	22/29
Warp	20	8.08×	12.87×	18/20
Resample	44	7.24×	5.65×	44/44
Raster I/O	58	4.00×	3.48×	56/58
Terrain	5	1.83×	1.91×	4/5
Viewshed	2	1.98×	2.02×	2/2

Per-category speedups over operations with GDAL CLI equivalents. Records are individual op × scale measurements, not unique operations. June 2026 run.

Categories represented by only a small number of GDAL-comparable records at production scale (Terrain, Viewshed) are reported for completeness; their operations are validated extensively elsewhere in the run against analytic and domain oracles, where the bulk of those categories' surface has no GDAL equivalent at all.

What is not claimed

ForgeGIS is positioned at parity with GDAL on format-conversion round-trips — full read-process-write cycles dominated by I/O rather than compute — and these are not counted as wins. ForgeGIS is also at parity on standalone single-operation invocations at very small raster scales (64² and 256²), where GDAL CLI process-spawn overhead is comparable to total ForgeGIS execution time and the comparison is not informative about underlying compute performance. The performance claims in this section apply at production raster scales (1024² and above) for catalog operations and multi-stage pipelines. The small number of head-to-head losses in the run occur entirely at these sub-production scales.

Validation and Coverage

ForgeGIS is engineered for the audit and reproducibility standards of regulated geospatial work. Every shipped operation has unit-test coverage, benchmark coverage, and a documented validation oracle. The June 2026 validation run executes 1,010 operation records spanning the catalog across multiple raster scales and fixture classes.

Validation oracles

Outputs are validated against domain-appropriate references rather than a single universal oracle, recognizing that no single reference covers the breadth of the catalog:

- **JTS (Java Topology Suite)** is the primary oracle for Cartesian predicates, topological relationships, and planar geometry operations.
- **PostGIS** is the oracle for spheroidal computations on the WGS84 ellipsoid where Cartesian-only oracles are insufficient.
- **GDAL CLI** is the comparison reference for raster operations with direct equivalents, scored through a tolerance-tiered framework (bit-exact, floating-point strict, structural-similarity, and per-operation continuous-output gates) applied per operation class.
- **Analytic numerical fixtures** — analytic surfaces, bump targets, ramp gradients, and synthesized inputs with known closed-form outputs — validate operations where no external oracle exists, particularly in terrain, bathymetry, and ML categories.

By internal convention, cuSpatial is recognized as the marketing-comparison peer in the GPU geospatial landscape but is intentionally not used as an oracle: where cuSpatial and JTS disagree (notably on edge-inclusion semantics and certain predicate corner cases), ForgeGIS sides with JTS.

Validation surface beyond GDAL

779 of the 1,010 validation records exercise operations with no GDAL CLI equivalent. This is the surface area on which ForgeGIS makes capability claims rather than performance claims — the operations that exist because organizations need them and that have historically required specialized tools (GRASS, SAGA, WhiteboxTools, TauDEM) outside the GDAL toolchain. The principal harness categories accounting for the bulk of this surface are terrain analytics, spatial-compute and topology, interpolation, zonal and focal analysis, spectral and ML classification, ocean analysis, hydrology, and viewshed.

Cross-surface determinism and reproducibility

The MCP server dispatches to the same compute implementations exposed through the direct Java API — a parameter-marshaling shim, not a parallel implementation. Most operations are deterministic by construction; the two operations involving randomness (K-means and random forest) run with a fixed deterministic default seed under MCP invocation, so agent runs are reproducible without per-call configuration, while the direct Java API exposes the seed as a parameter for callers that need to vary it. The validation harness verifies determinism by re-executing each operation and comparing outputs byte-for-byte; the June 2026 run reports the full record set passing this gate.

What this validation framework supports

The framework is designed so an evaluator can ask three independent questions and receive defensible answers: Is the operation correct? (oracle match), does it produce the same result when invoked by an agent as when invoked by a script? (surface-equivalence by construction), and has the validation surface kept pace with the catalog? (the coverage matrix). That separation is what audit and reproducibility requirements in regulated work demand.

Deployment and Integration

Reference hardware

All performance figures in this brief were measured on commodity laptop hardware: an Intel Core 7 240H (10C/16T) with 15.7 GB system RAM and an NVIDIA GeForce RTX 5070 Laptop GPU (8,150 MB VRAM, 36 compute units), running Windows 11, OpenJDK Corretto 26, GDAL 3.12.1, NVIDIA driver 573.22, and OpenCL 3.0 (CUDA 12.8.97), JVM heap max 4,096 MB. The reference platform is deliberately a single laptop — ForgeGIS does not require server-class infrastructure to deliver the performance characteristics documented here.

Software requirements

Java 17 or later. A GPU with OpenCL 1.2 or later support; NVIDIA, Intel, and AMD platforms are all targets. Apache Maven for build integration. No GDAL installation is required at runtime — the GDAL CLI dependency exists only in the benchmark harness for comparative measurement. The library is distributed as standard Maven artifacts under the com.seaglassfoundry group identifier; core compute, I/O, and MCP-server modules are independently consumable.

MCP server deployment

The MCP server ships as a single shaded executable jar, launched as a standalone process communicating over JSON-RPC on stdio per the MCP specification. Process-restart cold start (the typical operational case, with warm OS cache) is approximately 800 ms median to first tool-list response; first-deploy cold start (no OS file cache) is approximately 3 seconds. By default the server advertises the 8-tool MINIMAL surface; the full 68-tool surface is enabled with --profile standard or an environment variable.

Air-gapped and regulated deployment

Because ForgeGIS contains no first-party native code and no GDAL native bindings, deployment to air-gapped, classified, or otherwise restricted environments does not require coordinating separate native binary distribution, multi-source GLIBC validation, or shared-library signing for ForgeGIS itself. The deployment artifact is the same Maven jar (or shaded jar, for the standalone MCP server) that runs in unrestricted environments. The JOCL OpenCL bindings ship their JNI bridge bundled inside the JOCL jar; the system OpenCL runtime required for GPU dispatch is standard software bundled with GPU drivers. The required external components — JDK, GPU driver, OpenCL runtime — all have established procurement and approval paths in regulated environments.

Licensing and Contact

Licensing. ForgeGIS license terms are being finalized and will be published prior to general availability. Organizations with specific licensing requirements are invited to contact Seaglass Foundry directly.

Partnership and distribution. Seaglass Foundry is seeking strategic partners and distributors positioned to bring ForgeGIS to defense, intelligence, climate, and enterprise markets at scale. Inquiries from organizations with relevant distribution, integration, or capital-deployment capacity are welcome.

Contact. rich@seaglassfoundry.com · seaglassfoundry.com

About Seaglass Foundry

Seaglass Foundry LLC is an independent geospatial software company based in Panama City Beach, Florida, building the compute substrate for AI-native geospatial analysis. Its shipping product line includes ForgeGIS (GPU-accelerated geospatial library and MCP server) and SwingToPDF™ (vector PDF export for Java Swing applications, in production on Maven Central).

Appendix A — Complete Operation Catalog

[This appendix enumerates all 239 user-facing operations across the eleven categories, with each operation's canonical name, GDAL-CLI equivalent where one exists, and API surface (Java / MCP intent tool / pipeline DSL). It is pending op-for-op certification against the v1.1.0 source registries and will be populated in the final build.]

Appendix B — Benchmark Methodology

Run provenance. Benchmarks were executed by the forgegis-validation harness (commit 54373d54) against ForgeGIS v1.1.0 (release commit 6bf3f04, annotated tag v1.1.0), with the validation run taken immediately following the release tag. Both commit identifiers are recorded so each half of the result — the harness and the code under test — is independently reproducible.

Reference hardware. Intel Core 7 240H (10C/16T); 15.7 GB RAM; NVIDIA GeForce RTX 5070 Laptop GPU (8,150 MB VRAM, 36 compute units); Windows 11; OpenJDK Corretto 26 (26+35); NVIDIA driver 573.22; OpenCL 3.0 (CUDA 12.8.97); JVM heap max 4,096 MB. The harness used GDAL 3.12.1 as the comparison reference. Timing protocol: GC once per warm block (v2).

Run scope. 1,010 operation records across the catalog, organized into per-op category benchmarks, multi-stage pipeline benchmarks, and real-world dataset benchmarks. 779 of the records test operations with no GDAL CLI equivalent and use the validation-oracle conventions described in the Validation and Coverage section.

Headline cohort rule. The headline geometric mean is computed over every record whose reference is GDAL and where both backends were timed with a positive warm-mean — no category excluded. This yields a cohort of 225 records. Format round-trips whose GDAL reference is an untimed read-only check self-exclude through the both-backends-timed clause rather than through any category filter.

Timing methodology. Each measurement runs multiple warm iterations per operation per scale; head-to-head and per-category figures report warm-mean times (and warm-minimum where noted), excluding the cold first invocation that captures JIT warmup and OS-cache effects. Median and geometric-mean statistics are computed across the per-op series.

Raster scale strategy. Headline performance figures are drawn from production raster scales of 1024² and 4096², where compute time dominates and CLI process-spawn overhead is a small fraction of measured time. Scales of 64² and 256² are included in the full run for completeness and to disarm cherry-picking criticism, but are excluded from headline materials because at those scales GDAL CLI process-spawn overhead can dominate total ForgeGIS execution time, making the comparison non-informative about underlying compute performance.

Pipeline construction. The multi-stage pipeline benchmarks construct GDAL CLI chains using temporary GeoTIFF files between stages, the standard way these workflows are orchestrated outside an in-process compute library. ForgeGIS executes the same conceptual chain GPU-resident with no intermediate disk writes. This is a fair comparison of the operational shapes the two systems are designed for.

Verdicts. Operations with GDAL CLI equivalents are verdicted by structural similarity with appropriate tolerances per operation class. Operations without GDAL CLI equivalents are verdicted against the validation oracles described above. The full per-op verdict record is preserved in the run's JSONL output.

ForgeGIS, Seaglass Foundry, Seaglass Globe, and SwingToPDF are trademarks of Seaglass Foundry LLC. GDAL, JTS, PostGIS, OpenCL, Vulkan, NVIDIA, GeForce, RTX, CUDA, cuSpatial, QGIS, Intel, Microsoft, Windows, Java, OpenJDK, Apache, and Maven are trademarks or registered trademarks of their respective owners. Reference to these marks does not imply endorsement. © 2026 Seaglass Foundry LLC. All rights reserved.